

TP Kubernetes

SUJET :

Services Mesh sous Minikube

MEMBRES DU GROUPE :

Banconlé AKOBI
Luc DELAGE
Nathan ROUSSELLE
Lorick DESERT

Année universitaire 2025-2026

FISA 4 Informatique

Sommaire

Étape 1 : manipulations.....	2
Gestion de minikube.....	2
Q1 - Vérifier que minikube pointe vers docker.....	2
Q2 - Lister les addons installés.....	3
Q3 - Installer un addon (ex: metrics-server, utile pour monitorer les ressources CPU/RAM des pods).....	3
Q4 - Lister les profils avec toutes les caractéristiques.....	4
Q5 - Profils en cours.....	4
Q6 - Créer un nouveau profil.....	4
Q7 - Statut de minikube.....	5
Q8 - Accéder au dashboard.....	5
Q9 - Le dashboard de minikube.....	6
Q10 - Lister les nœuds :	6
Q11 - Ajouter puis supprimer un nœud :	6
Q12 - Consulter les logs :	8
Gestion Pods & Services sous Kubernetes.....	8
Q13 - Lister les images en exécution.....	8
Q14 - Lancer nginx :	9
Q15 - Exposer le service :	9
Q16 - Infos pod + service :	9
Q17 - URL du service :	11
Q18 - Ouvrir dans le navigateur :	11
Q19 - Lancer bash dans le conteneur nginx :	12
Q20 - Logs du conteneur :	12
Arrêter minikube.....	12
Questions Docker Build.....	13
Étape 2 : Développement d'un projet d'une architecture de type SERVICE.....	18
1. Architecture.....	18
2. Déploiement.....	19
3. Linkerd - Service Mesh.....	23
4. Manuel utilisateur.....	26
5. Manuel technique.....	28

Prérequis : démarrer minikube

```
minikube start --driver=docker
```

```
(banconle@mrbia)-[~]
$ minikube start --driver=docker
🐳 minikube v1.38.1 on Debian kali-rolling
🌟 Using the docker driver based on existing profile
👍 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.50 ...
🔄 Restarting existing docker container for "minikube" ...

⚠️ Docker is nearly out of disk space, which may cause deployments to fail! (87% of capacity). You can pass '--force' to skip this check.
💡 Suggestion:

Try one or more of the following to free up space on the device:

1. Run "docker system prune" to remove unused Docker data (optionally with "-a")
2. Increase the storage allocated to Docker for Desktop by clicking on:
   Docker icon > Preferences > Resources > Disk Image Size
3. Run "minikube ssh -- docker system prune" if using the Docker container runtime
🔗 Related issue: https://github.com/kubernetes/minikube/issues/9024

📦 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔍 Verifying Kubernetes components...
  • Using image docker.io/kubernetes/dashboard:v2.7.0
  • Using image gcr.io/k8s-minikube/storage-provisioner:v5
  • Using image docker.io/kubernetes/metrics-scraper:v1.0.8
💡 Some dashboard features require the metrics-server addon. To enable all features please run:
```

Étape 1 : manipulations

Gestion de minikube

Q1 - Vérifier que minikube pointe vers docker

```
$ minikube docker-env
```

```
(banconle@mrbia)-[~]
$ minikube docker-env
export DOCKER_TLS_VERIFY="1"
export DOCKER_HOST="tcp://192.168.49.2:2376"
export DOCKER_CERT_PATH="/home/banconle/.minikube/certs"
export MINIKUBE_ACTIVE_DOCKERD="minikube"

# To point your shell to minikube's docker-daemon, run:
# eval $(minikube -p minikube docker-env)

(banconle@mrbia)-[~]
```

Q2 - Lister les addons installés

```
$ minikube addons list
```

ADDON NAME	PROFILE	STATUS	MAINTAINER
ambassador	minikube	disabled	3rd party (Ambassador)
amd-gpu-device-plugin	minikube	disabled	3rd party (AMD)
auto-pause	minikube	disabled	minikube
cloud-spanner	minikube	disabled	Google
csi-hostpath-driver	minikube	disabled	Kubernetes
dashboard	minikube	enabled ✓	Kubernetes
default-storageclass	minikube	enabled ✓	Kubernetes
efk	minikube	disabled	3rd party (Elastic)
freshpod	minikube	disabled	Google
gcp-auth	minikube	disabled	Google
gvisor	minikube	disabled	minikube
headlamp	minikube	disabled	3rd party (kinvolk.io)
inaccel	minikube	disabled	3rd party (InAccel [info@inaccel.com])
ingress	minikube	disabled	Kubernetes
ingress-dns	minikube	disabled	minikube
inspektor-gadget	minikube	disabled	3rd party (inspektor-gadget.io)
istio	minikube	disabled	3rd party (Istio)
istio-provisioner	minikube	disabled	3rd party (Istio)
kong	minikube	disabled	3rd party (Kong HQ)
kubeflow	minikube	disabled	3rd party
kubetail	minikube	disabled	3rd party (kubetail.com)
kubevirt	minikube	disabled	3rd party (KubeVirt)
logviewer	minikube	disabled	3rd party (unknown)
metallb	minikube	disabled	3rd party (MetalLB)
metrics-server	minikube	disabled	Kubernetes
nvidia-device-plugin	minikube	disabled	3rd party (NVIDIA)
nvidia-driver-installer	minikube	disabled	3rd party (NVIDIA)
nvidia-gpu-device-plugin	minikube	disabled	3rd party (NVIDIA)
olm	minikube	disabled	3rd party (Operator Framework)
pod-security-policy	minikube	disabled	3rd party (unknown)
portainer	minikube	disabled	3rd party (Portainer.io)
registry	minikube	disabled	minikube
registry-aliases	minikube	disabled	3rd party (unknown)
registry-creds	minikube	disabled	3rd party (UPMC Enterprises)
storage-provisioner	minikube	enabled ✓	minikube
storage-provisioner-rancher	minikube	disabled	3rd party (Rancher)
volcano	minikube	disabled	third-party (volcano)
volumesnapshots	minikube	disabled	Kubernetes
yakd	minikube	disabled	3rd party (marcnuri.com)

Q3 - Installer un addon (ex: metrics-server, utile pour monitorer les ressources CPU/RAM des pods)

```
$ minikube addons enable metrics-server
```

```
(banconle@mrbia)-[~]
$ minikube addons enable metrics-server
🔔 metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  • Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
🌟 The 'metrics-server' addon is enabled
```

On active **metrics-server** parce qu'il permet de surveiller la consommation CPU et mémoire des pods avec *kubectl top*. Sans lui, impossible de savoir si un pod est en train de saturer ses ressources. C'est un des premiers trucs qu'on active à priori sur un cluster.

Q4 - Lister les profils avec toutes les caractéristiques

```
$ minikube profile list
```

```
(banconle@mrbia)-[~]
$ minikube profile list
```

PROFILE	DRIVER	RUNTIME	IP	VERSION	STATUS	NODES	ACTIVE PROFILE	ACTIVE KUBECONTEXT
minikube	docker	docker	192.168.49.2	v1.35.1	OK	1	*	*

On obtient un tableau avec le nom du profil, le driver utilisé (docker, virtualbox...), l'IP du noeud, la version de Kubernetes, le nombre de noeuds et le statut. C'est la vue d'ensemble du cluster.

Q5 - Profils en cours

```
(banconle@mrbia)-[~]
$ minikube profile list
```

PROFILE	DRIVER	RUNTIME	IP	VERSION	STATUS	NODES	ACTIVE PROFILE	ACTIVE KUBECONTEXT
minikube	docker	docker	192.168.49.2	v1.35.1	OK	1	*	*

Q6 - Créer un nouveau profil

```
minikube start -p TpArchi --driver=docker
```

Un profil, c'est un cluster Kubernetes isolé. Chaque profil a son propre noeud, sa propre config et sa propre version de Kubernetes. C'est pratique pour tester des configurations différentes sans casser l'environnement principal.

```
(banconle@mrbia)-[~]
$ minikube start -p TpArchi --driver=docker
[TpArchi] minikube v1.38.1 on Debian kali-rolling
Using the docker driver based on user configuration
Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
Using Docker driver with root privileges
Starting "TpArchi" primary control-plane node in "TpArchi" cluster
Pulling base image v0.0.50 ...
Creating docker container (CPUs=2, Memory=3900MB) ...

Docker is nearly out of disk space, which may cause deployments to fail! (90% of capacity). You can pass '--force' to skip this check.
Suggestion:

Try one or more of the following to free up space on the device:

1. Run "docker system prune" to remove unused Docker data (optionally with "-a")
2. Increase the storage allocated to Docker for Desktop by clicking on:
   Docker icon > Preferences > Resources > Disk Image Size
3. Run "minikube ssh -- docker system prune" if using the Docker container runtime
Related issue: https://github.com/kubernetes/minikube/issues/9024

Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
Configuring bridge CNI (Container Networking Interface) ...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: storage-provisioner, default-storageclass
Done! kubectrl is now configured to use "TpArchi" cluster and "default" namespace by default
```

Q7 - Statut de minikube

```
minikube status
```

```
(banconle@mrbia)-[~]
$ minikube status

minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

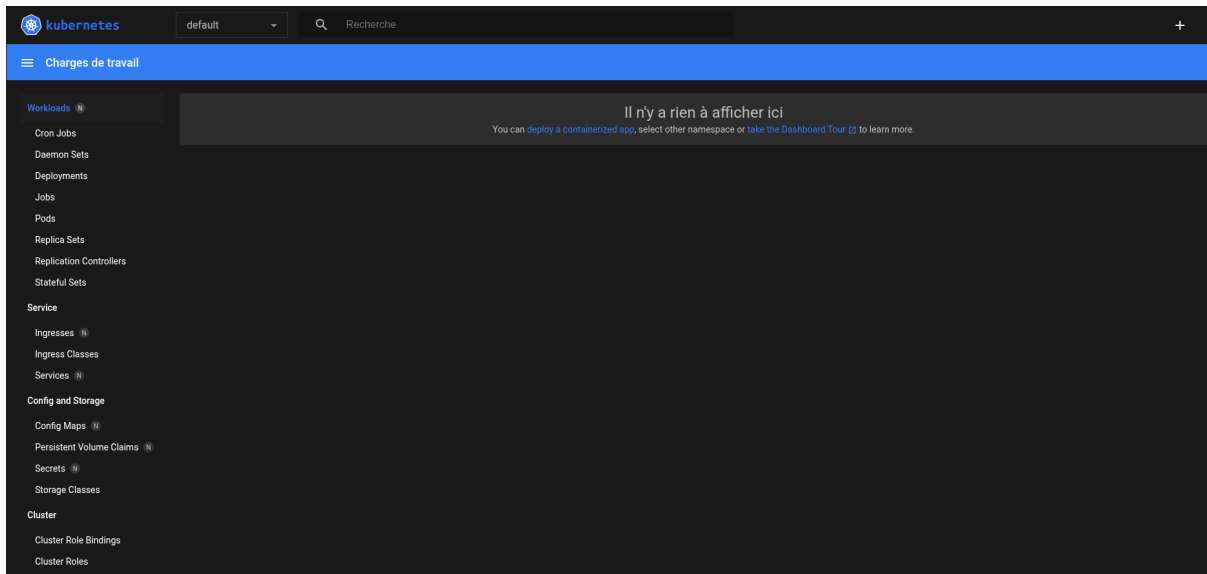
Q8 - Accéder au dashboard

```
minikube dashboard
```

```
(banconle@mrbia)-[~]
$ minikube dashboard

Verifying dashboard health ...
Launching proxy ...
Verifying proxy health ...
Opening http://127.0.0.1:36455/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/
in your default browser...
Opening in existing browser session.

Cron Jobs
Daemon Sets
```



Q9 - Le dashboard de minikube

Le Dashboard Kubernetes est une interface web qui donne une vue graphique de tout ce qui tourne dans le cluster : pods, deployments, services, namespaces, volumes, etc. On peut y voir l'état de santé des composants, consulter les logs, et même créer ou supprimer des ressources directement depuis le navigateur. C'est l'équivalent graphique de *kubectl*.

Q10 - Lister les nœuds :

```
minikube kubectl get nodes
```

```
(banconle@mrbia)-[~]
$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
tparchi    Ready    control-plane   96s   v1.35.1
```

Q11 - Ajouter puis supprimer un nœud :

```
minikube node add
```

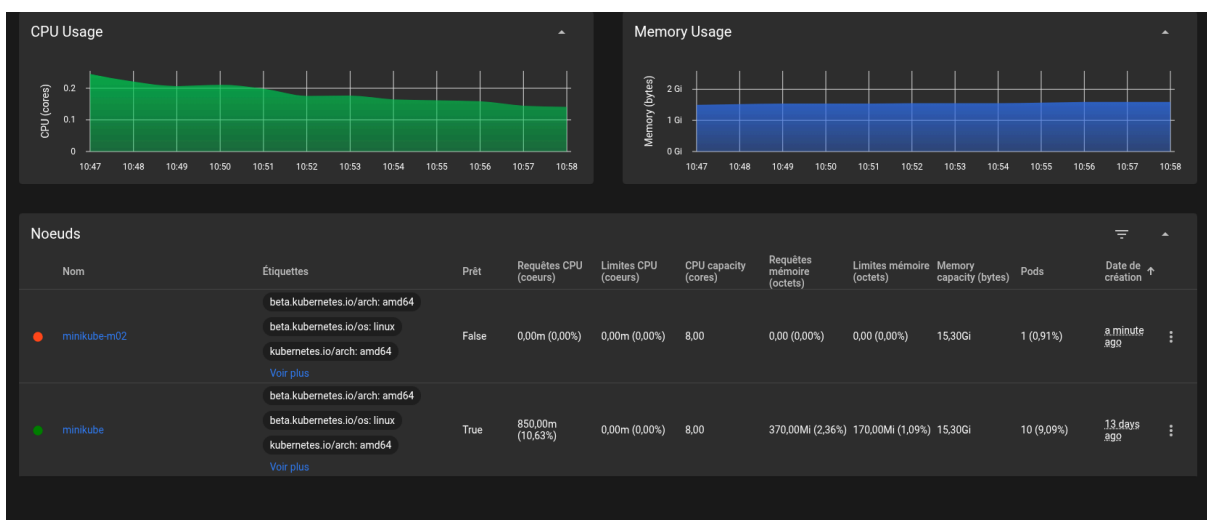
```
(banconle@mrbia)-[~]
$ minikube node add
Adding node m02 to cluster minikube as [worker]
Cluster was created without any CNI, adding a node to it might cause broken networking.
Starting "minikube-m02" worker node in "minikube" cluster
Pulling base image v0.0.50 ...
Creating docker container (CPUs=2, Memory=2200MB) ...

! Docker is nearly out of disk space, which may cause deployments to fail! (95% of capacity). You can pass '--force' to skip this check.
💡 Suggestion:

Try one or more of the following to free up space on the device:

1. Run "docker system prune" to remove unused Docker data (optionally with "-a")
2. Increase the storage allocated to Docker for Desktop by clicking on:
   Docker icon > Preferences > Resources > Disk Image Size
3. Run "minikube ssh -- docker system prune" if using the Docker container runtime
🔗 Related issue: https://github.com/kubernetes/minikube/issues/9024

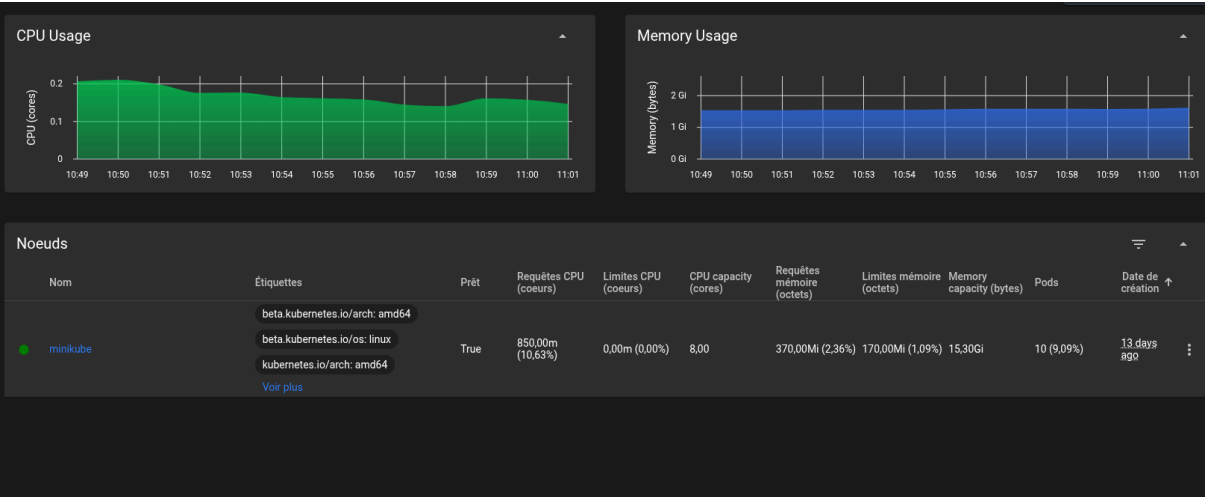
🐳 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
```



Pour le supprimer :

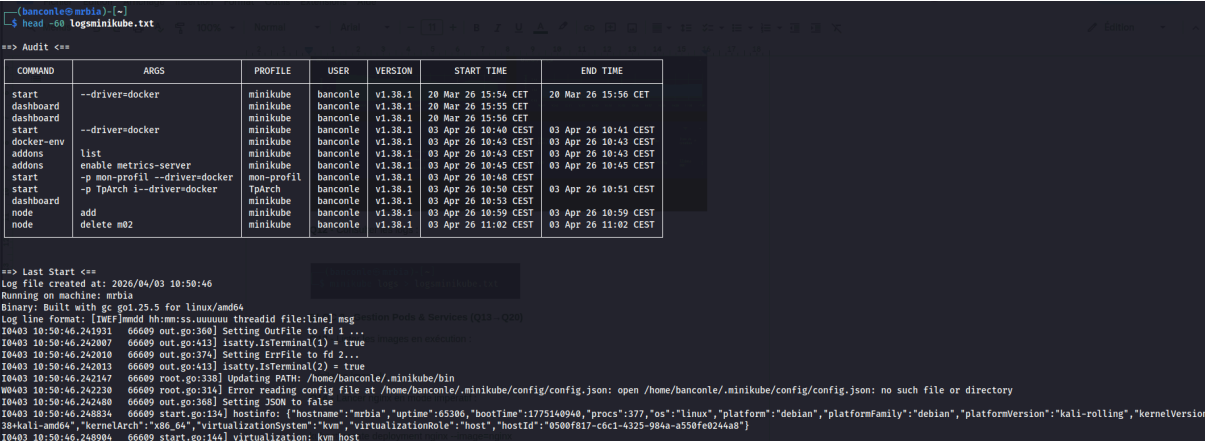
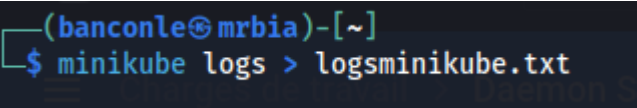
```
minikube node delete m02
```

```
(banconle@mrbia)-[~]
$ minikube node delete m02
Deleting node m02 from cluster minikube
Stopping node "minikube-m02" ...
Powering off "minikube-m02" via SSH ...
Deleting "minikube-m02" in docker ...
Node m02 was successfully deleted.
```

Q12 - Consulter les logs :

```
$ minikube logs > logsminikube.txt
```



Gestion Pods & Services sous Kubernetes

Q13 - Lister les images en exécution

```
minikube image ls
```

```
(banconle@mrbia)-[~]
$ minikube image ls
registry.k8s.io/pause:3.10.1
registry.k8s.io/kube-scheduler:v1.35.1
registry.k8s.io/kube-proxy:v1.35.1
registry.k8s.io/kube-controller-manager:v1.35.1
registry.k8s.io/kube-apiserver:v1.35.1
registry.k8s.io/etcd:3.6.6-0
registry.k8s.io/coredns/coredns:v1.13.1
gcr.io/k8s-minikube/storage-provisioner:v5
```

Q14 - Lancer nginx :

```
minikube kubectl -- createdeployment nginx -image=nginx
```

```
(banconle@mrbia)-[~]
$ minikube kubectl -- create deployment nginx --image=nginx

> kubectl.sha256: 64 B / 64 B [-----] 100.00% ? p/s 0s
> kubectl: 55.88 MiB / 55.88 MiB [-----] 100.00% 2.74 MiB p/s 21s
deployment.apps/nginx created
```

Q15 - Exposer le service :

```
minikube kubectl -- expose deployment nginx -type=NodePort -port=80
```

```
(banconle@mrbia)-[~]
$ minikube kubectl -- expose deployment nginx --type=NodePort --port=80

service/nginx exposed
```

Q16 - Infos pod + service :

```
minikube kubectl -- get pods -o wide
```

```
(banconle@mrbia)-[~]
$ minikube kubectl -- get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
nginx-56c45fd5ff-f2r7v	1/1	Running	0	104s	10.244.0.9	minikube	<none>	<none>

```

(banconle@mrbia)-[~]
$ minikube kubectl describe pod nginx-56c45fd5ff-f2r7v
Name: nginx-56c45fd5ff-f2r7v
Namespace: default
Priority: 0
Service Account: default
Node: minikube/192.168.49.2
Start Time: Fri, 03 Apr 2026 11:28:33 +0200
Labels: app=nginx
        pod-template-hash=56c45fd5ff
Annotations: <none>
Status: Running
IP: 10.244.0.9
IPs:
  IP: 10.244.0.9
Controlled By: ReplicaSet/nginx-56c45fd5ff
Containers:
  nginx:
    Container ID: docker://cbb35c7b150f823737473e7ec171d66355f1a703eb70947e469b94ad5fd9b27a
    Image: nginx
    Image ID: docker-pullable://nginx@sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
    Port: <none>
    Host Port: <none>
    State: Running
      Started: Fri, 03 Apr 2026 11:28:43 +0200
      Ready: True
      Restart Count: 0
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-zpn6q (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized         True

```

```

(banconle@mrbia)-[~]
$ minikube kubectl -- get svc nginx
NAME      TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
nginx     NodePort  10.109.34.117   <none>            80:32198/TCP     3m3s

```

```
(banconle@mrbia)-[~]
$ minikube kubectl -- describe svc nginx

Name:          nginx
Namespace:     default
Labels:        app=nginx
Annotations:    <none>
Selector:      app=nginx
Type:          NodePort
IP Family Policy: SingleStack
IP Families:   IPv4
IP:            10.109.34.117
IPs:           10.109.34.117
Port:          <unset> 80/TCP
TargetPort:    80/TCP
NodePort:      <unset> 32198/TCP
Endpoints:     10.244.0.9:80
Session Affinity: None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events:        <none>

(banconle@mrbia)-[~]
```

Q17 - URL du service :

```
minikube service nginx --url
```

```
(banconle@mrbia)-[~]
$ minikube service nginx --url

http://192.168.49.2:32198
```

Q18 - Ouvrir dans le navigateur :

```
minikube service nginx
```

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

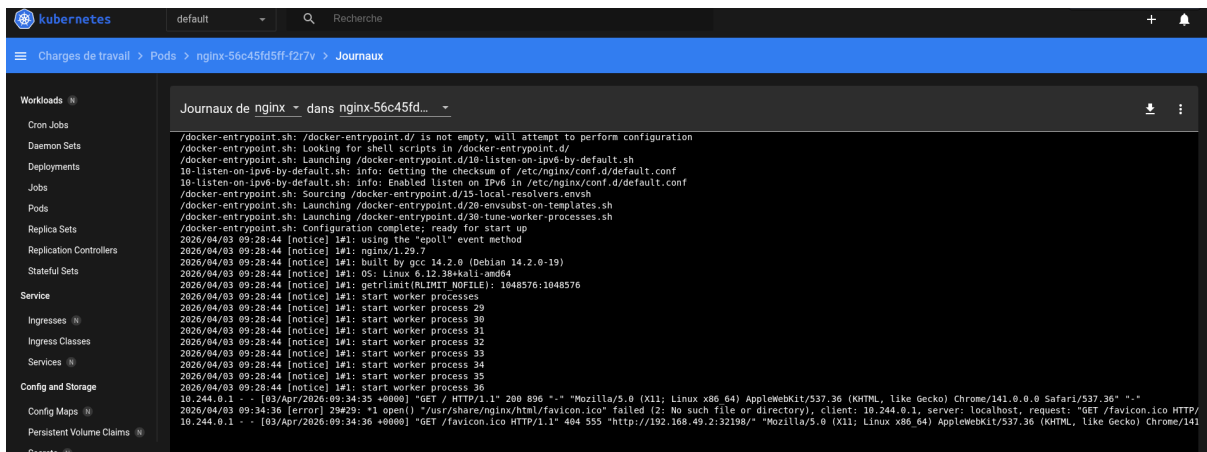
Thank you for using nginx.

Q19 - Lancer bash dans le conteneur nginx :

```
(banconle@mrbia)-[~]  
$ minikube kubectl -- exec -it nginx-56c45fd5ff-f2r7v -- /bin/bash  
  
root@nginx-56c45fd5ff-f2r7v:/# ls
```

Q20 - Logs du conteneur :

```
(banconle@mrbia)-[~]  
$ minikube kubectl logs nginx-56c45fd5ff-f2r7v  
docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration  
docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/  
docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh  
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf  
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf  
docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh  
docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh  
docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh  
docker-entrypoint.sh: Configuration complete; ready for start up  
2026/04/03 09:28:44 [notice] 1#1: using the "epoll" event method  
2026/04/03 09:28:44 [notice] 1#1: nginx/1.29.7  
2026/04/03 09:28:44 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)  
2026/04/03 09:28:44 [notice] 1#1: OS: Linux 6.12.38+kali-amd64  
2026/04/03 09:28:44 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576  
2026/04/03 09:28:44 [notice] 1#1: start worker processes  
2026/04/03 09:28:44 [notice] 1#1: start worker process 29  
2026/04/03 09:28:44 [notice] 1#1: start worker process 30  
2026/04/03 09:28:44 [notice] 1#1: start worker process 31  
2026/04/03 09:28:44 [notice] 1#1: start worker process 32  
2026/04/03 09:28:44 [notice] 1#1: start worker process 33  
2026/04/03 09:28:44 [notice] 1#1: start worker process 34  
2026/04/03 09:28:44 [notice] 1#1: start worker process 35  
2026/04/03 09:28:44 [notice] 1#1: start worker process 36  
0.244.0.1 - - [03/Apr/2026:09:34:35 +0000] "GET / HTTP/1.1" 200 896 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36" "-"  
2026/04/03 09:34:36 [error] 29#29: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 10.244.0.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "192.168.49.2:32198", referer: "http://192.168.49.2:32198/"  
0.244.0.1 - - [03/Apr/2026:09:34:36 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://192.168.49.2:32198/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36" "-"  
(banconle@mrbia)-[~]
```



Arrêter minikube

minikube stop

Questions Docker Build

https://github.com/BanconleAkobi/Kubernetes_Services_Mesh_sous_Minikube/tree/Etape1

Questions Docker Build

Préparation

Le web service Spring Boot expose deux endpoints :

```
GET /monservice/echo/{nom}
POST /monservice/hello avec un body JSON {"nom":"valeur"}
```

Compilation du fat-JAR et test local

On compile le fat-JAR avec Maven :

```
mvn clean package -DskipTests
```

```
l-$ mvn clean package -DskipTests
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.insta:tp_archi_step_1 >-----
[INFO] Building 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[WARNING] Parameter 'fork' is unknown for plugin 'spring-boot-maven-plugin:4.0.5:repackage (repackage)'
[WARNING] Parameter 'fork' is unknown for plugin 'spring-boot-maven-plugin:4.0.5:repackage (default)'
[INFO] --- clean:3.5.0:clean (default-clean) @ tp_archi_step_1 ---
[INFO] Deleting /home/banconle/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1/target
[INFO] --- resources:3.3.1:resources (default-resources) @ tp_archi_step_1 ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO] --- compiler:3.14.1:compile (default-compile) @ tp_archi_step_1 ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 2 source files with javac [debug parameters release 21] to target/classes
[INFO] --- resources:3.3.1:testResources (default-testResources) @ tp_archi_step_1 ---
[INFO] skip non existing resourceDirectory /home/banconle/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1/src/test/resources
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ tp_archi_step_1 ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug parameters release 21] to target/test-classes
[INFO] --- surefire:3.5.5:test (default-test) @ tp_archi_step_1 ---
[INFO] Tests are skipped.
[INFO] --- jar:3.4.2:jar (default-jar) @ tp_archi_step_1 ---
[INFO] Building jar: /home/banconle/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1/target/tp_archi_step_1-0.0.1-SNAPSHOT.jar
[INFO] --- spring-boot:4.0.5:repackage (repackage) @ tp_archi_step_1 ---
[INFO] Replacing main artifact /home/banconle/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1/target/tp_archi_step_1-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to /home/banconle/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1/target/tp_archi_step_1-0.0.1-SNAPSHOT.jar.original
[INFO] --- spring-boot:4.0.5:repackage (default) @ tp_archi_step_1 ---
[INFO] Replacing main artifact /home/banconle/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1/target/tp_archi_step_1-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to /home/banconle/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1/target/tp_archi_step_1-0.0.1-SNAPSHOT.jar.original
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.883 s
[INFO] Finished at: 2026-05-02T16:49:45+02:00
[INFO] -----
```

```
java -jar target/tp_archi_step_1-0.0.1-SNAPSHOT.jar
```

Test des endpoints :

```
(banconle@mrbia)-[~/../java/com/insa/tp_archi_step_1]
$ curl "http://localhost:8080/monservice/echo?nom=Banconle"
{"message":"Echo : Banconle"}

(banconle@mrbia)-[~/../java/com/insa/tp_archi_step_1]
$ curl -X POST http://localhost:8080/monservice/hello -H "Content-Type: application/json" -d '{"nom":"Banconlé"}'
{"message":"Hello Banconlé !"}

```

Le Dockerfile :

14


```
WORKDIR /app
COPY target/tp_archi_step_1-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Le docker-compose.yml :

```
version: "3.8"
services:
  tp_archi_step_1:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    container_name: tp_archi_step_1-container
```

```
docker compose up --build
```

[illegible]

Test du service conteneurisé :


```
curl -X POST http://localhost:8080/monservice/hello -H "Content-Type: application/json" -d '{"nom":"Banconlé"}'
```

```
(banconle@mrbia) - [~/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1]
$ curl -X POST http://localhost:8080/monservice/hello -H "Content-Type: application/json" -d '{"nom":"Banconlé"}'
{"message":"Hello Banconlé !"}
```

Cas 2 : Dockerfile multi-stage (sans compilation préalable)

Le Dockerfile.multistage compile les sources directement dans Docker. La première étape télécharge les dépendances et compile avec Maven, la deuxième ne garde que le JAR dans une image légère :

```
# Cas 1
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package
```

```
# cas 2
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /build/target/tp_archi_step_1-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Le docker-compose.multistage.yml :

```
version: "3.8"
services:
  tp_archi_step_1:
    build:
      context: .
      dockerfile: Dockerfile.multistage
    ports:
      - "8080:8080"
    container_name: tp_archi_step_1-multistage
```

```
docker compose -f docker-compose.multistage.yml up --build
```

```

(banconle@mrba) ~/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1
$ docker compose -f docker-compose.multistage.yml up --build

JARN[0000] /home/banconle/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1/docker-compose.multistage.yml: the attribute
'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 110.1s (16/16) FINISHED
=> [tp_archi_step_1 internal] load build definition from Dockerfile.multistage                                docker:default
=> => transferring dockerfile: 397B                                                                    0.0s
=> [tp_archi_step_1 internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine                0.5s
=> [tp_archi_step_1 internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-21                2.0s
=> [tp_archi_step_1 internal] load .dockerignore                                                            0.0s
=> => transferring context: 2B                                                                              0.0s
=> [tp_archi_step_1 builder 1/6] FROM docker.io/library/maven:3.9-eclipse-temurin-21@sha256:8e7dc4215c70f922 23.5s
=> => resolve docker.io/library/maven:3.9-eclipse-temurin-21@sha256:8e7dc4215c70f922e798c9f8aafa0a3734ca38634 0.0s
=> => sha256:8e7dc4215c70f922e798c9f8aafa0a3734ca386342427b3dbb17cecc4a429c8e 6.65kB / 6.65kB          0.0s
=> => sha256:f192d0389be99807761e2aa70e0fecba8f33918e5762d268ba6643f487eed1e6 2.73kB / 2.73kB          0.0s
=> => sha256:8a14c71c9922c7b7ace98d670546a5f0504c287182ca9ef4d95c6fad4d66dc83 9.24kB / 9.24kB          0.0s
=> => sha256:b40150c1c2717d324cdb17278c8efd4d4dcd2ffe083e976f0bcdcf31115f081 29.73MB / 29.73MB        5.5s
=> => sha256:a2c74f5c2809456d4ead8df7e9a8c6ce210cd1a451308fd3ee42cd5613fac2f 22.97MB / 22.97MB        5.9s
=> => sha256:3b9a1e97487bbf567fd39c1a971160f21dc69bfea8ee8bfc923b39fe9c05f8d 157.87MB / 157.87MB      18.7s
=> => extracting sha256:b40150c1c2717d324cdb17278c8efd4d4dcd2ffe083e976f0bcdcf31115f081                1.2s
=> => sha256:2ef33ef1b9930d5fb66257dbd8f7e1ccb258d35b87243481902922fddcfcae86 156B / 156B              5.9s
=> => sha256:e9d29f6a4c60335d154318f22c76be42011017740ad6c05fdb6d56ee283f3030 2.28kB / 2.28kB          6.3s
=> => sha256:3a0cfe6cd7e3ea307af3c977e7587353fbdde9c79b6195c14f68ff7e9f914d84 22.54MB / 22.54MB      10.1s
=> => sha256:c708c0a14e8a983cac9b61f9722467ff85661198c365529d9f54f54bc61cbeee 9.31MB / 9.31MB          8.5s
=> => extracting sha256:a2c74f5c2809456d4ead8df7e9a8c6ce210cd1a451308fd3ee42cd5613fac2f                0.9s
=> => sha256:b552d4a3844f33860c2d9492a88e3c76a8135878a304a0971b888144bda2e33b 852B / 852B              8.9s
=> => sha256:1396908a28e98cde78ac7a8c848b953c7b0f1c2ccdc64efa4eec120b5c8f6f44 156B / 156B              9.4s
=> => extracting sha256:3b9a1e97487bbf567fd39c1a971160f21dc69bfea8ee8bfc923b39fe9c05f8d                3.4s
=> => extracting sha256:2ef33ef1b9930d5fb66257dbd8f7e1ccb258d35b87243481902922fddcfcae86                0.0s
=> => extracting sha256:e9d29f6a4c60335d154318f22c76be42011017740ad6c05fdb6d56ee283f3030                0.0s
=> => extracting sha256:3a0cfe6cd7e3ea307af3c977e7587353fbdde9c79b6195c14f68ff7e9f914d84                1.0s
=> => extracting sha256:c708c0a14e8a983cac9b61f9722467ff85661198c365529d9f54f54bc61cbeee                0.2s
=> => extracting sha256:b552d4a3844f33860c2d9492a88e3c76a8135878a304a0971b888144bda2e33b                0.0s
=> => extracting sha256:1396908a28e98cde78ac7a8c848b953c7b0f1c2ccdc64efa4eec120b5c8f6f44                0.0s
=> [tp_archi_step_1 internal] load build context                                                            0.0s
=> => transferring context: 4.04KB                                                                            0.0s
=> [tp_archi_step_1 stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:ad0cdd9782db550c 0.0s
=> [tp_archi_step_1 builder 2/6] WORKDIR /build                                                            0.3s
=> [tp_archi_step_1 builder 3/6] COPY pom.xml .                                                            0.0s
=> [tp_archi_step_1 builder 4/6] RUN mvn dependency:go-offline -B                                           70.3s
=> [tp_archi_step_1 builder 5/6] COPY src ./src                                                            0.0s
=> [tp_archi_step_1 builder 6/6] RUN mvn clean package                                                    13.8s
=> CACHED [tp_archi_step_1 stage-1 2/3] WORKDIR /app                                                       0.0s
=> [tp_archi_step_1 stage-1 3/3] COPY --from=builder /build/target/tp_archi_step_1-0.0.1-SNAPSHOT.jar app.jar 0.0s
=> [tp_archi_step_1] exporting to image                                                                    0.1s
=> => exporting layers                                                                                        0.1s
=> => writing image sha256:05d672698cc2d98008ee642ca6290a5ce3af577332247474bbb5d55ffe270ff                0.0s
=> => naming to docker.io/library/tp_archi_step_1-tp_archi_step_1                                          0.0s
=> [tp_archi_step_1] resolving provenance for metadata file                                                0.0s
[+] Running 2/2
✔ tp_archi_step_1 Built 0.0s
✔ Container tp_archi_step_1-container Recreated 0.0s
Attaching to tp_archi_step_1-multistage

```

Test :

```
curl -X POST http://localhost:8080/monservice/hello -H "Content-Type: application/json" -d '{"nom":"Banconlé"}'
```

```

(banconle@mrba) ~/Documents/FISA_S8/TP_Kubernetes/tp_archi_step_1
$ curl -X POST http://localhost:8080/monservice/hello -H "Content-Type: application/json" -d '{"nom":"Banconlé"}'

{"message":"Hello Banconlé !"}

```

Intérêt du multi-stage

Avec le cas 1, il faut compiler le JAR en local avant de construire l'image — donc avoir Java et Maven installés. Avec le multi-stage, Docker fait tout en une passe : le mvn dependency:go-offline télécharge les dépendances, le mvn clean package compile, et seul le JAR est copié dans l'image finale. N'importe qui peut builder le projet avec juste Docker. L'image finale est aussi plus légère puisqu'elle ne contient que le JRE Alpine et le JAR.

Étape 2 : Développement d'un projet d'une architecture de type SERVICE

https://github.com/BanconleAkobi/Kubernetes_Services_Mesh_sous_Minikube/tree/Etape2_Merged

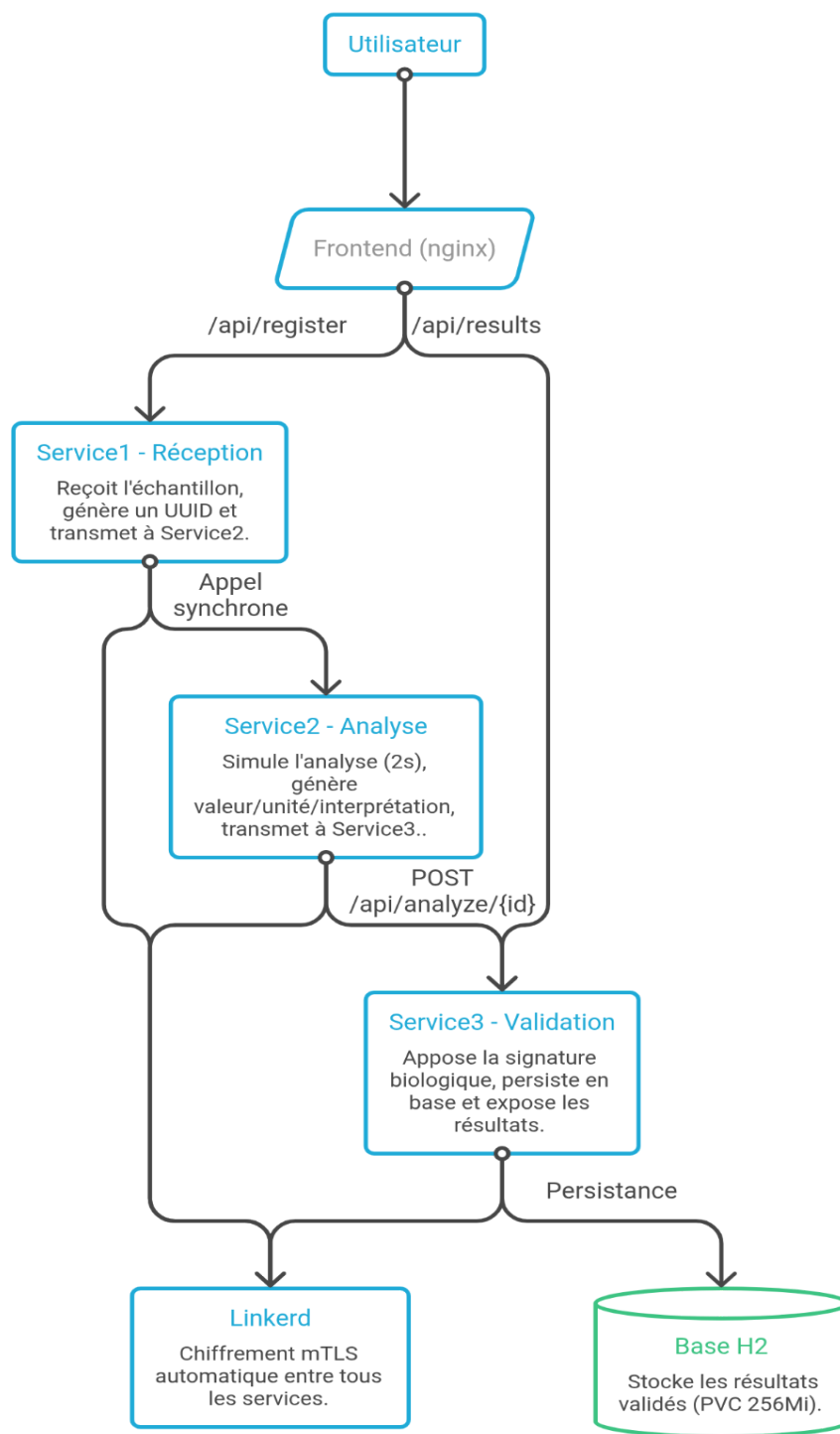
1. Architecture

LaboTrack simule le parcours d'un échantillon biologique dans un laboratoire d'analyses médicales, de la réception du prélèvement jusqu'à la validation du résultat.

L'application repose sur 4 services :

- **service1** (port 9000): réception. Prend les infos du prélèvement (patient, examen, type d'échantillon), génère un UUID et transmet la demande à service2.
- **service2** (port 9001): analyse. Simule un traitement de 2 secondes, génère une valeur, une unité et une interprétation (Normal, High, Low, Positive, Negative), puis transmet à service3.
- **service3** (port 9002): validation. Appose une signature biologique de 10 caractères, persiste le résultat en base H2. Expose les endpoints de consultation par UUID et l'historique complet.
- **frontend** (nginx): interface web avec un stepper en 3 étapes. nginx fait le proxy vers les backends.

Le flux : utilisateur → frontend → service1 → service2 → service3 → frontend affiche le résultat.



2. Déploiement

Le script `runbook.sh` automatise l'ensemble du déploiement. Les images sont taggées **labotrack/<service>:1.0**.

Démarrage de Minikube

```
./runbook.sh minikube-start
```

```
(banconle@mrbia)-[~/Documents/FISA_S8/TP_Kubernetes/Etape2_Merged]
$ ./runbook.sh minikube-start
>>> Démarrage Minikube...
🐳 minikube v1.38.1 on Debian kali-rolling
! minikube skips various validations when --force is supplied; this may lead to unexpected behavior
🌟 Automatically selected the docker driver
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
🔧 Using Docker driver with root privileges
👉 Starting "minikube" primary control-plane node in "minikube" cluster
📡 Pulling base image v0.0.50 ...
🔥 Creating docker container (CPUs=4, Memory=4096MB) ...
! Failing to connect to https://registry.k8s.io/ from both inside the minikube container and host machine
💡 To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/netwo
rking/proxy/
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
  • Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass
🏁 Done! kubectrl is now configured to use "minikube" cluster and "default" namespace by default
```

Installation de Linkerd

```
./runbook.sh linkerd-install
```

```
(banconle@mrbia)-[~/Documents/FISA_S8/TP_Kubernetes/Etape2_Merged]
$ ./runbook.sh linkerd-install

>>> Vérification pré-installation Linkerd...
kubernetes-api
-----
! can initialize the client
! can query the Kubernetes API

kubernetes-version
-----
! is running the minimum Kubernetes API version

pre-kubernetes-setup
-----
! control plane namespace does not already exist
! can create non-namespaced resources
! can create ServiceAccounts
! can create Services
! can create Deployments
! can create CronJobs
! can create ConfigMaps
! can create Secrets
! can read Secrets
! can read extension-apiserver-authentication configmap
! no clock skew detected

linkerd-version
-----
! can determine the latest version
!! cli is up-to-date
  is running version 26.4.4 but the latest edge version is 26.5.1
  see https://linkerd.io/2/checks/#l5d-version-cli for hints
```

Cette commande enchaîne : le pré-check, l'installation des Gateway API CRDs, les CRDs Linkerd, le control plane, et Linkerd Viz.

```

linkerd-config
-----
✓ control plane Namespace exists
✓ control plane ClusterRoles exist
✓ control plane ClusterRoleBindings exist
✓ control plane ServiceAccounts exist
✓ control plane CustomResourceDefinitions exist
✓ control plane MutatingWebhookConfigurations exist
✓ control plane ValidatingWebhookConfigurations exist
✓ proxy-init container runs as root user if docker container runtime is used

linkerd-identity
-----
✓ certificate config is valid
✓ trust anchors are using supported crypto algorithm
✓ trust anchors are within their validity period
✓ trust anchors are valid for at least 60 days
✓ issuer cert is using supported crypto algorithm
✓ issuer cert is within its validity period
✓ issuer cert is valid for at least 60 days
✓ issuer cert is issued by the trust anchor

linkerd-webhooks-and-apisvc-tls
-----
✓ proxy-injector webhook has valid cert
✓ proxy-injector cert is valid for at least 60 days
✓ sp-validator webhook has valid cert
✓ sp-validator cert is valid for at least 60 days
✓ policy-validator webhook has valid cert
✓ policy-validator cert is valid for at least 60 days

linkerd-version
-----
✓ can determine the latest version
!! cli is up-to-date
   is running version 26.4.4 but the latest edge version is 26.5.1
   see https://linkerd.io/2/checks/#l5d-version-cli for hints

control-plane-version
-----
✓ can retrieve the control plane version
!! control plane is up-to-date
   is running version 26.4.4 but the latest edge version is 26.5.1
   see https://linkerd.io/2/checks/#l5d-version-control for hints
✓ control plane and cli versions match

linkerd-control-plane-proxy
-----
✓ control plane proxies are healthy
!! control plane proxies are up-to-date
   some proxies are not running the current version:
   * linkerd-destination-69bc5c84bc-mbv9g (edge-26.4.4)
   * linkerd-identity-597c79667-79lnn (edge-26.4.4)
   * linkerd-proxy-injector-6844c7bc78-kl67s (edge-26.4.4)
   see https://linkerd.io/2/checks/#l5d-cp-proxy-version for hints

```

```

-----
✓ linkerd-viz Namespace exists
✓ can initialize the client
✓ linkerd-viz ClusterRoles exist
✓ linkerd-viz ClusterRoleBindings exist
✓ tap API server has valid cert
✓ tap API server cert is valid for at least 60 days
✓ tap API service is running
✓ linkerd-viz pods are injected
✓ viz extension pods are running
✓ viz extension proxies are healthy
!! viz extension proxies are up-to-date
   some proxies are not running the current version:
   * metrics-api-644dbd4584-hk9ds (edge-26.4.4)
   * prometheus-fd9b79d75-rl8bb (edge-26.4.4)
   * tap-5dd75b9c5f-p79xf (edge-26.4.4)
   * tap-injector-76bd74bdf-c9kcf (edge-26.4.4)
   * web-86bc477f8d-8tv2f (edge-26.4.4)
   see https://linkerd.io/2/checks/#l5d-viz-proxy-cp-version for hints
✓ viz extension proxies and cli versions match
✓ prometheus is installed and configured correctly
✓ viz extension self-check

Status check results are ✓

```

Build des images

```
./runbook.sh build
```

```
(banconte@mrblia) - [~/Documents/FISA_S8/TP_Kubernetes/Etape2_Merged]
$ ./runbook.sh build
>>> Connexion au daemon Docker de Minikube...
>>> Build service1...
[+] Building 147.1s (15/15) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                 0.0s
=> => transferring dockerfile: 340B                                0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 6.2s
=> [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-21 6.3s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
```

Les images sont buildées directement dans le daemon Docker de Minikube (eval \$(minikube docker-env)), donc pas besoin de push vers un registre.

```
(banconte@mrblia) - [~/Documents/FISA_S8/TP_Kubernetes/Etape2_Merged]
$ docker images | grep labotrack
labotrack/frontend          1.0          d5370c588da8  2 minutes ago  48.3MB
labotrack/service3         1.0          2fe427cf9142  2 minutes ago  265MB
labotrack/service2         1.0          d7d18538ff99  4 minutes ago  229MB
labotrack/service1         1.0          ff17f0133c61  5 minutes ago  265MB
```

Déploiement de l'application

```
./runbook.sh deploy
```

Les manifests sont appliqués dans l'ordre : namespace, service3, service2, service1, frontend. Le script attend que chaque deployment soit prêt avec kubectl rollout status.

```
$ ./runbook.sh deploy
>>> Namespace labotrack...
namespace/labotrack created
>>> Déploiement des services...
persistentvolumeclaim/service3-data created
deployment.apps/service3 created
service/service3 created
deployment.apps/service2 created
service/service2 created
serviceprofile.linkerd.io/service2.labotrack.svc.cluster.local created
deployment.apps/service1 created
service/service1 created
deployment.apps/frontend created
service/frontend created
>>> Attente des pods...
Waiting for deployment "service3" rollout to finish: 0 of 1 updated replicas are available...
deployment "service3" successfully rolled out
deployment "service2" successfully rolled out
Waiting for deployment "service1" rollout to finish: 0 of 1 updated replicas are available...
deployment "service1" successfully rolled out
deployment "frontend" successfully rolled out
>>> Pods :
NAME                                READY   STATUS    RESTARTS   AGE
frontend-d575ff749-5zd9c            2/2     Running   0           25s
service1-7d8ffbdf66-2lbn6           2/2     Running   0           25s
service2-587846f75-57dk6           2/2     Running   0           26s
service3-64559fd5b7-6wg4x          2/2     Running   0           26s
```


Accès au frontend

```
./runbook.sh open
```

```
L$ ./runbook.sh open

>>> URL du frontend :
http://192.168.49.2:30080
```

Lancement complet en une commande

Pour tout faire depuis zéro (Minikube → Linkerd → build → deploy) :

```
./runbook.sh all
```

```
L$ ./runbook.sh all
>>> Démarrage Minikube...
🐹 minikube v1.38.1 on Debian kali-rolling
  • MINIKUBE_ACTIVE_DOCKERD=minikube
  ! minikube skips various validations when --force is supplied; this may lead to unexpected behavior
  ! Automatically selected the docker driver
  ! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
  🚀 Using Docker driver with root privileges
  🏠 Starting "minikube" primary control-plane node in "minikube" cluster
  📡 Pulling base image v0.0.50 ...
  🔥 Creating docker container (CPUs=4, Memory=4096MB) ...
  🚢 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
  ⚙️ Configuring bridge CNI (Container Networking Interface) ...
  🔍 Verifying Kubernetes components...
  • Using image gcr.io/k8s-minikube/storage-provisioner:v5
  ⭐ Enabled addons: storage-provisioner, default-storageclass
  🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
>>> Vérification pré-installation Linkerd...
kubernetes-api
-----
✓ can initialize the client
✓ can query the Kubernetes API

kubernetes-version
-----
✓ is running the minimum Kubernetes API version

pre-kubernetes-setup
-----
✓ control plane namespace does not already exist
✓ can create non-namespaced resources
✓ can create ServiceAccounts
✓ can create Services
✓ can create Deployments
✓ can create CronJobs
✓ can create ConfigMaps
✓ can create Secrets
✓ can read Secrets
```

3. Linkerd - Service Mesh

mTLS

Linkerd chiffre automatiquement tout le trafic entre services avec du mutual TLS.

```
linkerd viz edges -n labotrack
```

[capture]

Métriques

```
linkerd viz stat deploy -n labotrack
```

```
(banconle@mrbia)-[~/Documents/FISA_S8/TP_Kubernetes/Etape2_Merged]
$ linkerd viz stat deploy -n labotrack
```

NAME	MESHED	SUCCESS	RPS	LATENCY_P50	LATENCY_P95	LATENCY_P99	TCP_CONN
frontend	1/1	100.00%	0.3rps	1ms	5ms	5ms	1
service1	1/1	100.00%	0.4rps	1ms	18ms	20ms	2
service2	1/1	100.00%	0.4rps	1ms	18ms	20ms	2
service3	1/1	100.00%	0.3rps	1ms	9ms	10ms	1

Trafic en temps réel

```
linkerd viz top deploy -n labotrack
```

```
banconle@mrbia: ~/Documents/FISA_S8/TP_Kubernetes/Etape2_Merged 117x40
(linkerd viz top deploy -n labotrack)
(please q to quit)
(please a/LeftArrowKey to scroll left, d/RightArrowKey to scroll right)
```

Source	Destination	Method	Path
10.244.0.1	service2-587846f75-77clh	GET	/actuator/health
10.244.0.1	service3-64559fd5b7-mbtnm	GET	/actuator/health
10.244.0.1	service1-7d8ffbfd66-xvsxp	GET	/actuator/health
frontend-d575ff749-ds5xz	service1-7d8ffbfd66-xvsxp	POST	/api/register
service2-587846f75-77clh	service3-64559fd5b7-mbtnm	POST	/api/validate
10.244.0.1	frontend-d575ff749-ds5xz	POST	/api/register
service1-7d8ffbfd66-xvsxp	service2-587846f75-77clh	POST	/api/analyze
frontend-d575ff749-ds5xz	service3-64559fd5b7-mbtnm	GET	/api/validate/validated/61ddc3f5-0896-4cb9-9757-4
10.244.0.1	frontend-d575ff749-ds5xz	GET	/api/results/61ddc3f5-0896-4cb9-9757-499b46ae725c

Requêtes en live

```
linkerd viz tap deploy/service1 -n labotrack
```

```
(banconle@mrbia)-[~/Documents/FISA_S8/TP_Kubernetes]
$ linkerd viz tap deploy/service1 -n labotrack

req id=1:0 proxy=in src=10.244.0.1:37704 dst=10.244.0.13:9000 tls=no_tls_from_remote :method=GET :authority=10.244.0.13:9000 :path=/actuator/health
rsp id=1:0 proxy=in src=10.244.0.1:37704 dst=10.244.0.13:9000 tls=no_tls_from_remote :status=200 latency=15903µs
end id=1:0 proxy=in src=10.244.0.1:37704 dst=10.244.0.13:9000 tls=no_tls_from_remote duration=2098µs response-length=0B
req id=1:1 proxy=in src=10.244.0.14:58500 dst=10.244.0.13:9000 tls=true :method=POST :authority=192.168.49.2 :path=/api/register
req id=1:2 proxy=in src=10.244.0.1:37644 dst=10.244.0.13:9000 tls=no_tls_from_remote :method=GET :authority=10.244.0.13:9000 :path=/actuator/health
rsp id=1:2 proxy=in src=10.244.0.1:37644 dst=10.244.0.13:9000 tls=no_tls_from_remote :status=200 latency=8110µs
```

ServiceProfile

Le ServiceProfile de service2 configure un timeout de 15 secondes sur POST /api/analyze et désactive les retries (l'opération n'est pas idempotente).

```
kubectl get serviceprofile -n labotrack
```

```
(banconle@mrbia)-[~/Documents/FISA_S8/TP_Kubernetes]
$ kubectl get serviceprofile -n labotrack

NAME                                AGE
service2.labotrack.svc.cluster.local 3m
```

Dashboard

```
./runbook.sh check
```

Namespaces						
HTTP Metrics						
Namespace	Meshed	Success Rate	RPS	P50 Latency	P95 Latency	P99 Latency
default	0/0	---	---	---	---	---
kube-node-lease	0/0	---	---	---	---	---
kube-public	0/0	---	---	---	---	---
kube-system	0/7	---	---	---	---	---
labotrack	4/4	100.00% ●	1.5	1 ms	16 ms	19 ms
linkerd	3/3	100.00% ●	2.67	1 ms	9 ms	17 ms
linkerd-viz	5/5	100.00% ●	3.73	3 ms	18 ms	26 ms
TCP Metrics						
Namespace	Meshed	Connections	Read Bytes / sec	Write Bytes / sec		
default	0/0	---	---	---		
kube-node-lease	0/0	---	---	---		
kube-public	0/0	---	---	---		
kube-system	0/7	---	---	---		

4. Manuel utilisateur

Utilisation

1. Remplir le formulaire : nom du patient, type d'examen, type de prélèvement.

The screenshot displays the LaboTrack web application interface. At the top, a dark blue header contains the LaboTrack logo, the text 'Système de suivi des analyses de laboratoire', and a 'Historique' button. Below the header, a progress bar shows three steps: 1. ENREGISTREMENT (active), 2. ANALYSE, and 3. RÉSULTAT. The main content area features a white card titled 'NOUVEL ÉCHANTILLON'. Inside the card, there is a 'PATIENT' section with a text input field containing 'Carlos'. Below this, there are two dropdown menus: 'TYPE D'EXAMEN' with 'Glycémie' selected, and 'TYPE DE PRÉLÈVEMENT' with 'Sang' selected. A blue button labeled 'Enregistrer l'échantillon' is positioned at the bottom right of the card.

2. L'analyse se lance automatiquement (2 secondes de traitement).

LaboTrack

Système de suivi des analyses de laboratoire

Historique

1

2

3

ENREGISTREMENTANALYSERÉSULTAT

ÉCHANTILLON ENREGISTRÉ

IDd48db10e-a3ee-4421-8b04-2d989d18e8f6

PATIENTCarlos

EXAMENGLYCEMIE

PRÉLÈVEMENTSANG

Lancer l'analyse

3. Le résultat s'affiche : valeur, unité, interprétation et signature de validation.

RÉSULTAT DE L'ANALYSE

84.98 g/L

POSITIVE

PATIENTCarlos

EXAMENGLYCEMIE

VALIDATION BIOLOGIQUEe92CTLP9fx

Nouvel échantillon

4. Le bouton « Historique » affiche tous les résultats passés.

HISTORIQUE DES ANALYSES

PATIENT	EXAMEN	PRÉLÈVEMENT	VALEUR	INTERPRÉTATION	SIG
Carlos	GLYCEMIE	SANG	84.98 g/L	POSITIVE	e92
Banconlé	NFS	URINE	23.46 mmol/L	POSITIVE	5yo
Luc	BILAN_HEPATIQUE	PLASMA	46.93 g/L	NORMAL	A0F

Autres commandes

```
./runbook.sh compose-up      # test local via Docker Compose
./runbook.sh compose-down    # arrête Docker Compose
./runbook.sh check           # état Linkerd + stats
```

Arrêt

```
kubectl delete namespace labotrack
minikube stop
```

5. Manuel technique

Stack

Les trois backends sont en Java 21 / Spring Boot 4.x avec Maven, Lombok, MapStruct et JPA. La base de données est H2 en mode fichier (service3 uniquement, persistée via un PVC de 256Mi). Le frontend est en JavaScript vanilla servi par nginx 1.27 Alpine.

Conteneurisation

Chaque service backend a un Dockerfile multi-stage : compilation Maven dans une première image, exécution JRE Alpine dans la deuxième. Le frontend utilise un entrypoint avec envsubst pour injecter les URLs backend (SERVICE1_URL, SERVICE3_URL) dans la config nginx au runtime. Les images sont taggées labotrack/<service>:1.0.

Kubernetes

Le namespace labotrack a l'annotation linkerd.io/inject: enabled; tous les pods reçoivent automatiquement le sidecar proxy. Les deployments utilisent imagePullPolicy: Never puisque les images sont buildées dans le daemon Minikube. Chaque backend a un

readinessProbe sur /actuator/health. Service3 a un PersistentVolumeClaim de 256Mi monté sur /data.

Linkerd

Les Gateway API CRDs sont installés avant le control plane. Le mTLS est automatique entre tous les services. Un ServiceProfile sur service2 configure un timeout de 15s et désactive les retries.